

American Basket Option Pricing: Single Level and Multi-Level Experiments

Sebastian Lalvay

April 8, 2026

This report summarizes the main implementation experiments on Single-Level and Multi-Level methods for American Basket Option Pricing using Markovian Projections. It is based on jupyter notebooks located in the main repo: `L2_Regression/notebooks_seb`

1. `nb04_Legendre_single_level` introduces the example and apply single level method.
2. `nb05_Legendre_Multi_level` explore the Multi Level method for the same example.

nb04_Legendre_single_level

April 8, 2026

```
[2]: # --- Setup: base imports ---
import sys
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import inspect
import html
from IPython.display import display, HTML

# --- Setup: project path ---
PROJECT_ROOT = Path.cwd().parent
if str(PROJECT_ROOT) not in sys.path:
    sys.path.append(str(PROJECT_ROOT))

# Add Single_Level folder as well, because some files import
# sibling modules with 'from SL_legendre_utilities import ...'
SINGLE_LEVEL_DIR = PROJECT_ROOT / "Legendre_Polynomials" / "Single_Level"
if str(SINGLE_LEVEL_DIR) not in sys.path:
    sys.path.append(str(SINGLE_LEVEL_DIR))

# Optional (recommended) for syntax highlighting
from pygments import highlight
from pygments.lexers import PythonLexer
from pygments.formatters import HtmlFormatter

_PYGMENTS_CSS_LOADED = False

def show_full_function(func, name=None):
    global _PYGMENTS_CSS_LOADED

    name = name or func.__name__
    source = inspect.getsource(func)

    formatter = HtmlFormatter(style="friendly", noclasses=False)
    highlighted = highlight(source, PythonLexer(), formatter)
```

```

if not _PYGMENTS_CSS_LOADED:
    css = formatter.get_style_defs('.highlight')
    display(HTML(f"<style>{css}</style>"))
    _PYGMENTS_CSS_LOADED = True

html_block = f"""
<details>
    <summary><strong>View full function: {html.escape(name)}</strong></
↪summary>
    {highlighted}
</details>
"""
display(HTML(html_block))

from Legendre_Polynomials.Single_Level.SL_legendre_utilities import (
    GBM_paths,
    scalings_l0,
    tot_degree_poly,
    normaleq_components_SL,
    fit_local_vol,
    make_b_bar,
)

from Legendre_Polynomials.Single_Level.SL_surface_visualisation import (
    plot_local_volatility,
)

from Legendre_Polynomials.Single_Level.SL_distribution_validation import (
    validate_log_returns,
)

from Legendre_Polynomials.Single_Level.SL_weak_error_analysis import (
    compute_weak_error,
    plot_convergence_study,
    export_results_csv,
)

```

What this Notebook will do

This notebook integrates the whole **Single_Level** workflow of the Legendre folder:

1. build the Legendre regression model from `SL_legendre_utilities.py`,
2. fit the projected volatility function,
3. visualize the local volatility surface,
4. validate the terminal distribution of the Markovian projection,
5. study weak error convergence for option pricing.

Single-Level Markovian Projection with Legendre Polynomial Regression

0.1 Black-Scholes Correlated Model:

We consider a d -dimensional correlated Black-Scholes model

$$dX(t) = a(t, X(t)) dt + b(t, X(t)) dW(t), \quad 0 \leq t \leq T, \quad X(t) \in \mathbb{R}^d, \quad (1)$$

$$X(0) = x_0. \quad (2)$$

where - $W(t)$ is a k -dim. Brownian motion. - $a(t, x) = r x$, $r = 0.05$.

- $b(t, x) \in \mathbb{R}^{d \times k}$.

- $(b_{BS}(t, x))_{ij} = x_i \Sigma_{ij}$.

0.1.1 Example ($d = 3, k = 3$)

Let $x = (x_1, x_2, x_3)^\top$ and $\sigma = (\sigma_1, \sigma_2, \sigma_3)^\top$. Assume the correlated Black-Scholes diffusion is defined by

$$b_{BS}(t, x) = \text{diag}(x) \text{diag}(\sigma) G, \quad \text{equivalently } (b_{BS}(t, x))_{ij} = \sigma_i x_i G_{ij}$$

where,

$$\Sigma_{ij} = \sigma_i G_{ij}, \quad GG^\top = \begin{pmatrix} 1 & 0.8 & 0.3 \\ 0.8 & 1 & 0.1 \\ 0.3 & 0.1 & 1 \end{pmatrix}, \quad \sigma = (0.2, 0.15, 0.1)^\top.$$

Each component satisfies

$$dX_i(t) = r X_i(t) dt + \sigma_i X_i(t) \sum_{j=1}^k G_{ij} dW_j(t), \quad i = 1, \dots, d.$$

Thus, even if the Brownian motions $W_1, \dots, W_k$ are independent, the assets are correlated through the matrix G .

0.2 Markovian Projection

Let $S(t) = P_1 X(t)$, with $P_1 \in \mathbb{R}^{1 \times d}$ and $X(t) \in \mathbb{R}^{d \times 1}$. Define the Markovian surrogate $\bar{S}^{(x_0)}(t)$ by

$$\begin{cases} d\bar{S}^{(x_0)}(t) = \bar{a}^{(x_0)}(t, \bar{S}^{(x_0)}(t)) dt + \bar{b}^{(x_0)}(t, \bar{S}^{(x_0)}(t)) d\bar{W}(t), & t \in [0, T], \\ \bar{S}^{(x_0)}(0) = P_1 x_0. \end{cases}$$

The drift and diffusion coefficients are given by conditional expectations:

$$\bar{a}^{(x_0)}(t, s) = \mathbb{E}[P_1 a(t, X(t)) \mid P_1 X(t) = s, X(0) = x_0] = \mathbb{E}[r P_1 X(t) \mid P_1 X(t) = s, X(0) = x_0] = r s,$$

and

$$(\bar{b}^{(x_0)}(t, s))^2 = \mathbb{E}\left[P_1 b_{BS}(t, X(t)) b_{BS}(t, X(t))^\top P_1^\top \mid P_1 X(t) = s, X(0) = x_0\right].$$

For $P_1 = \frac{1}{d}(1, \dots, 1)$ and $C = GG^\top$, this becomes

$$(\bar{b}^{(x_0)}(t, s))^2 = \mathbb{E}\left[\frac{1}{d^2} \sum_{i=1}^d \sum_{j=1}^d \sigma_i \sigma_j X_i(t) X_j(t) C_{ij} \mid P_1 X(t) = s, X(0) = x_0\right],$$

0.3 L^2 Regression Problem

We approximate $(\bar{b}^{x_0})^2(\cdot, \cdot)$ by solving the time-integrated L^2 problem

$$(\bar{b}^{x_0})^2(\cdot, \cdot) \approx \arg \min_{h \in V} \int_0^T \mathbb{E} \left[\left(P b_{\text{BS}}(t, X(t)) b_{\text{BS}}(t, X(t))^\top P^\top - h(t, PX(t)) \right)^2 \mid X(0) = x_0 \right] dt.$$

For example, one may choose a parametric class

$$V = \{ h : h(t, x) = c_0 + c_1 t + c_2 x \}.$$

Discretizing time with a grid $0 = t_0 < t_1 < \dots < t_N = T$, an empirical Monte Carlo objective is

$$(\bar{b}^{x_0})^2(\cdot, \cdot) \approx \arg \min_{h \in V} \frac{1}{M} \sum_{m=1}^M \frac{1}{N} \sum_{n=1}^N \left(\frac{1}{d^2} \sum_{i=1}^d \sum_{j=1}^d \sigma_i \sigma_j X_i^{(m)}(t_n) X_j^{(m)}(t_n) C_{ij} - h(t_n, PX^{(m)}(t_n)) \right)^2.$$

Ansatz. We approximate the squared projected diffusion by a finite expansion

$$\tilde{b}^2(t, s) \approx \sum_{p=1}^P c_p \psi_p(t, s).$$

0.4 Legendre Polynomials

Legendre polynomials $\{P_n(x)\}_{n \geq 0}$ are orthogonal on $[-1, 1]$:

$$\int_{-1}^1 P_m(x) P_n(x) dx = \frac{2}{2n+1} \delta_{mn}.$$

The first few polynomials are: - $P_0(x) = 1$, - $P_1(x) = x$, - $P_2(x) = \frac{1}{2}(3x^2 - 1)$, - $P_3(x) = \frac{1}{2}(5x^3 - 3x)$.

0.4.1 Orthonormalization

Define the orthonormalized Legendre polynomials by

$$\tilde{P}_n(x) := \sqrt{\frac{2n+1}{2}} P_n(x).$$

Then

$$\int_{-1}^1 \tilde{P}_m(x) \tilde{P}_n(x) dx = \delta_{mn}.$$

0.5 Scaling of time and basket value

Because Legendre polynomials are naturally defined on $[-1, 1]$, both variables are scaled to that interval.

Time is mapped by

$$\tau(t) = \frac{2t}{T} - 1.$$

The basket variable is mapped by

$$\xi(s) = \frac{2(s - s_{\min})}{s_{\max} - s_{\min}} - 1.$$

The bounds $s_{\min}$ and $s_{\max}$ are obtained from a pilot simulation, using empirical lower and upper percentiles of the basket distribution. This gives a practical domain on which the Legendre approximation is fitted.

0.5.1 Basis Functions

We define bivariate basis functions using orthonormal Legendre polynomials on $[-1, 1]$:

$$\psi_{(i_1, i_2)}(t, s) = \tilde{P}_{i_1}(\tau(t)) \tilde{P}_{i_2}(\xi(s)), \quad p = (i_1, i_2) \in \Lambda.$$

We use a **total-degree** index set

$$\Lambda = \{(i_1, i_2) \in \mathbb{N}_0^2 : i_1 + i_2 \leq d_{\max}\}.$$

Examples. - $d_{\max} = 0$: $(i_1, i_2) = (0, 0) \Rightarrow 1$ basis function (constant). - $d_{\max} = 1$: $(0, 0), (0, 1), (1, 0) \Rightarrow 3$ basis functions. - $d_{\max} = 2$: $(0, 0), (0, 1), (0, 2), (1, 0), (2, 0), (1, 1) \Rightarrow 6$ basis functions.

The approximation of the projected variance is therefore

$$\tilde{b}^2(t, s) = \sum_{(i, j) \in \Lambda} c_{ij} \tilde{P}_i(\tau(t)) \tilde{P}_j(\xi(s)).$$

Finally, the projected local volatility is obtained by

$$\bar{b}(t, s) = \sqrt{\tilde{b}^2(t, s)}.$$

Reformulated L^2 Regression Problem Using the basis $\{\psi_p\}_{p \in \Lambda}$ defined above, we restrict the search space to the finite-dimensional class

$$V := \left\{ h : h(t, s) = \sum_{p \in \Lambda} c_p \psi_p(t, s), \quad c \in \mathbb{R}^{|\Lambda|} \right\}.$$

The discrete L^2 regression problem for the projected diffusion becomes

$$(\bar{b}^{x_0})^2(\cdot, \cdot) \approx \arg \min_{h \in V} \frac{1}{M} \frac{1}{N} \sum_{m=1}^M \sum_{n=1}^N \left(\frac{1}{d^2} \sum_{i=1}^d \sum_{j=1}^d \sigma_i \sigma_j X_i^{(m)}(t_n) X_j^{(m)}(t_n) C_{ij} - \sum_{p \in \Lambda} c_p \psi_p(t_n, PX^{(m)}(t_n)) \right)^2.$$

This transforms the problem into a **linear least-squares** problem for the coefficient vector $c = (c_p)_{p \in \Lambda}$.

Implementation

Implementation — Step 1: Define the market setting, choose the Legendre basis, and estimate the scaling domain

In this first step, we initialize the model and simulation parameters, define the Legendre basis used for the regression, and estimate the basket-value interval on which the approximation will be built.

The underlying model is the correlated Black–Scholes basket

$$dX_i(t) = rX_i(t) dt + \sigma_i X_i(t) (G dW(t))_i, \quad i = 1, \dots, d,$$

with equal-weight basket

$$S(t) = P_1^\top X(t), \quad P_1 = \frac{1}{d}(1, \dots, 1)^\top.$$

The approximation space is built from orthonormal Legendre polynomials, so both time and basket value must eventually be scaled to the interval $[-1, 1]$.

To do that, we need three ingredients: 1. a simulation of the basket model, 2. a choice of polynomial basis, 3. an estimate of the basket domain $[s_{\min}, s_{\max}]$ from a pilot run.

The first ingredient is the simulation of the high-dimensional process.

0.5.2 Function used: `GBM_paths`

Syntax

```
paths = GBM_paths(x0, r, vol, cov_mat, dt, N_t, M_t)
```

What it does - Simulates M_t paths of the correlated d -dimensional geometric Brownian motion.
- Uses the Cholesky factor of the correlation matrix to generate correlated Brownian increments. -
Returns an array of shape (M_t, N_t, d) containing all simulated asset paths.

Mathematically, this function generates Monte Carlo samples of

$$dX_i(t) = rX_i(t) dt + \sigma_i X_i(t) (G dW(t))_i.$$

The second ingredient is the choice of the Legendre basis.

0.5.3 Function used: `tot_degree_poly`

Syntax

```
pairs = tot_degree_poly(max_deg)
```

What it does - Generates the set of basis index pairs (i, j) satisfying

$$i + j \leq d_{\max}.$$

- This defines a **total-degree** tensor-product Legendre basis. - It is more economical than the full tensor-product basis, because it avoids including all combinations independently up to degree $d_{\max}$.

This means the approximation space is built from

$$\psi_{(i,j)}(t, s) = \widetilde{P}_i(\tau(t)) \widetilde{P}_j(\xi(s)), \quad i + j \leq d_{\max}.$$

The third ingredient is the scaling interval for the basket variable.

0.5.4 Function used: `scalings_10`

Syntax

```
s_min0, s_max0, basket0 = scalings_10(x0, T, dt, r, cov_mat, vol, P1, M_0=10000)
```

What it does - Performs a pilot simulation of the basket process. - Internally, it calls `GBM_paths(...)` to simulate many training paths. - Then it computes empirical lower and upper percentile bounds for the basket values. - Returns: - `s_min0`: lower basket bound, - `s_max0`: upper basket bound, - `basket0`: simulated basket paths from the pilot run.

These values are used to define the spatial scaling

$$\xi(s) = \frac{2(s - s_{\min})}{s_{\max} - s_{\min}} - 1.$$

So in summary, this step prepares the regression domain: - `GBM_paths` generates the stochastic data, - `tot_degree_poly` defines the Legendre basis, - `scalings_10` determines the basket interval on which the Legendre approximation will be fitted.

```
[4]: show_full_function(GBM_paths)
```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

```
[5]: show_full_function(tot_degree_poly)
```

<IPython.core.display.HTML object>

```
[6]: show_full_function(scalings_10)
```

<IPython.core.display.HTML object>

```
[12]: # Step 1: market setting + Legendre basis + pilot scaling
```

```
# Basket configuration
```

```
d = 3
```

```
P1 = np.ones(d) / d
```

```
# Market parameters
```

```
r = 0.05
```

```
x0 = np.linspace(225, 275, num=d)[: , np.newaxis]
```

```
vol = np.array([0.2, 0.15, 0.1])
```

```
cov_mat = np.array([
```

```
    [1.0, 0.8, 0.3],
```

```
    [0.8, 1.0, 0.1],
```

```
    [0.3, 0.1, 1.0]
```

```
])
```

```
# Time discretisation
```



```

T = 1.0
dt = 0.005
N_t = int(T / dt)
M_t = 400

t = np.linspace(0, T, N_t)

# Legendre basis choice
max_deg = 2
pairs = tot_degree_poly(max_deg)

# Pilot run for scaling domain
s_min0, s_max0, basket0 = scalings_l0(
    x0=x0,
    T=T,
    dt=dt,
    r=r,
    cov_mat=cov_mat,
    vol=vol,
    P1=P1,
    M_0=10000
)

print("\n--- Step 1 summary ---")
print("d =", d)
print("P1 =", P1)
print("max_deg =", max_deg)
print("number of basis functions =", len(pairs))
print("first basis pairs =", pairs[:6])
print("s_min0 =", s_min0)
print("s_max0 =", s_max0)
print("basket0 shape =", basket0.shape)
print("time grid shape =", t.shape)

```

--- Step 1 summary ---

```

d = 3
P1 = [0.33333333 0.33333333 0.33333333]
max_deg = 2
number of basis functions = 6
first basis pairs = [(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (2, 0)]
s_min0 = 182.77292450496537
s_max0 = 390.09352875967477
basket0 shape = (10000, 200)
time grid shape = (200,)

```

Implementation — Step 2: Generate training paths and build the Legendre regression system

In this step, we move from the pilot-domain setup to the actual regression data used to fit the projected variance.

The goal is to build the discrete least-squares problem

$$Dc \approx \psi,$$

where: - D is the Legendre design matrix, - ψ is the vector of true instantaneous basket variances, - c is the coefficient vector of the Legendre approximation.

To construct this system, we first generate a collection of training paths of the correlated Black–Scholes model, and then evaluate both: - the basis functions, - and the target projected variance, at each simulated path-time point.

0.5.5 Function used: GBM_paths

Syntax

`paths = GBM_paths(x0, r, vol, cov_mat, dt, N_t, M_t)`

What it does - Simulates M_t paths of the correlated d -dimensional geometric Brownian motion. - Uses the Cholesky factor of the correlation matrix to generate correlated Brownian increments. - Returns an array of shape (M_t, N_t, d) .

This gives the Monte Carlo sample

$$\{X^{(m)}(t_n)\}_{m=1,\dots,M_t, n=0,\dots,N_t-1}.$$

These paths are the data on which the regression system is constructed.

0.5.6 Function used: normaleq_components_SL

Syntax

`D, psi = normaleq_components_SL(paths, P1, pairs, cov_mat, vol, s_min0, s_max0, T)`

What it does - Computes the basket values

$$S^{(m)}(t_n) = P_1^\top X^{(m)}(t_n),$$

and scales them to the interval $[-1, 1]$ using

$$\xi(s) = \frac{2(s - s_{\min})}{s_{\max} - s_{\min}} - 1.$$

- Scales time using

$$\tau(t) = \frac{2t}{T} - 1.$$

- Evaluates the orthonormal Legendre basis functions on all simulated points. - Builds the design matrix

$$D \in \mathbb{R}^{(M_t N_t) \times P},$$

where each column corresponds to one basis pair (i, j) and each row corresponds to one simulated observation. - Computes the response vector

$$\psi \in \mathbb{R}^{M_t N_t},$$

whose entries are the true instantaneous basket variances.

For the equal-weight basket, the target at each simulated point is

$$\psi_{m,n} = \frac{1}{d^2} \sum_{i=1}^d \sum_{j=1}^d \sigma_i \sigma_j X_i^{(m)}(t_n) X_j^{(m)}(t_n) \Sigma_{ij}.$$

0.5.7 Mathematical meaning of this step

This step converts the stochastic simulation into a regression problem.

The fitted projected variance is sought in the finite-dimensional space

$$\tilde{b}^2(t, s) = \sum_{(i,j) \in \Lambda} c_{ij} \tilde{P}_i(\tau(t)) \tilde{P}_j(\xi(s)).$$

So: - the matrix D stores the basis evaluations, - the vector ψ stores the true variance targets, - and the next step will solve for the coefficient vector c .

```
[10]: show_full_function(normaleq_components_SL)
```

<IPython.core.display.HTML object>

```
[13]: # Step 2: generate training paths and build the regression system
```

```
paths = GBM_paths(x0, r, vol, cov_mat, dt, N_t, M_t)

D, psi = normaleq_components_SL(
    paths=paths,
    P1=P1,
    pairs=pairs,
    cov_mat=cov_mat,
    vol=vol,
    s_min=s_min0,
    s_max=s_max0,
    T=T
)

print("\n--- Step 2 summary ---")
print("paths shape =", paths.shape)
print("D shape =", D.shape)
print("psi shape =", psi.shape)
print("first basis pairs =", pairs[:6])
print("first row of D =", D[0])
print("first entries of psi =", psi[:5].ravel())
```

```
--- Step 2 summary ---
paths shape = (400, 200, 3)
D shape = (80000, 6)
```

```

psi shape = (80000, 1)
first basis pairs = [(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (2, 0)]
first row of D = [ 0.5          -0.30437978 -0.35185189 -0.8660254   0.52720124
1.11803399]
first entries of psi = [870.69444444 879.31055791 901.72826568 887.2723045
889.69466798]

```

Step 3: Solve the Legendre regression problem and construct the projected volatility function

Once the regression system has been built, we now solve the discrete least-squares problem

$$\min_{c \in \mathbb{R}^P} \|Dc - \psi\|_2^2,$$

where:

- D is the Legendre design matrix,
- ψ is the vector of target basket variances,
- c is the vector of Legendre coefficients.

The fitted projected variance is therefore

$$\tilde{b}^2(t, s) = \sum_{(i,j) \in \Lambda} c_{ij} \tilde{P}_i(\tau(t)) \tilde{P}_j(\xi(s)),$$

where

$$\tau(t) = \frac{2t}{T} - 1, \quad \xi(s) = \frac{2(s - s_{\min})}{s_{\max} - s_{\min}} - 1.$$

The projected local volatility is then defined by

$$\bar{b}(t, s) = \sqrt{\tilde{b}^2(t, s)}.$$

0.5.8 Function used: `fit_local_vol`

Syntax

```
c = fit_local_vol(D, psi)
```

What it does - Solves the least-squares regression problem using QR decomposition. - If

$$D = QR,$$

then the coefficient vector is obtained from

$$Rc = Q^\top \psi.$$

- Returns the fitted coefficient vector `c`.

This is the numerical step that identifies the best approximation of the projected variance inside the chosen Legendre basis.

0.5.9 Function used: make_b_bar

Syntax

```
b_bar = make_b_bar(c, pairs, s_min0, s_max0, T, max_deg)
```

What it does - Builds a callable function `b_bar(t, S)` from the fitted coefficients. - Rescales the input variables (t, S) to the interval $[-1, 1]$. - Evaluates the orthonormal Legendre expansion. - Returns

$$\bar{b}(t, s) = \sqrt{\tilde{b}^2(t, s)}.$$

Thus, this function transforms the regression coefficients into the actual local volatility function used in the projected one-dimensional SDE

$$d\bar{S}_t = r\bar{S}_t dt + \bar{b}(t, \bar{S}_t) dW_t.$$

0.5.10 Mathematical role of this step

This step is where the fitted model is finally created.

- Step 1 defined the basis and scaling domain.
- Step 2 built the regression system.
- Step 3 solves for the coefficients and converts them into a usable function.

From this point onward, everything in the notebook will use the single object

$$\bar{b}(t, s),$$

namely: - volatility surface visualization, - distribution validation, - weak error analysis.

```
[12]: show_full_function(fit_local_vol)
```

```
<IPython.core.display.HTML object>
```

```
[13]: show_full_function(make_b_bar)
```

```
<IPython.core.display.HTML object>
```

```
[14]: # Step 3: solve the regression problem and construct b_bar
```

```
c = fit_local_vol(D, psi)
b_bar = make_b_bar(c, pairs, s_min0, s_max0, T, max_deg)

print("\n--- Step 3 summary ---")
print("c shape =", c.shape)
print("number of coefficients =", len(c))
print("first coefficients =", c[:6])
print("b_bar callable created =", callable(b_bar))

# Quick sanity checks
test_points = [
    (t[0], float(np.mean(paths[:, 0, :].dot(P1)))),
```

```

        (t[len(t)//2], float(np.mean(paths[:, len(t)//2, :].dot(P1)))),
        (t[-1], float(np.mean(paths[:, -1, :].dot(P1))))
    ]

    for t_test, s_test in test_points:
        print(f"b_bar({t_test:.3f}, {s_test:.3f}) = {b_bar(t_test, s_test):.6f}")

```

--- Step 3 summary ---

```

c shape = (6,)
number of coefficients = 6
first coefficients = [ 2.59965157e+03  1.25178223e+03  1.73278973e+02
-1.82607609e+01
 -8.62534701e+00 -1.80695040e-01]
b_bar callable created = True
b_bar(0.000, 250.000) = 29.477195
b_bar(0.503, 256.731) = 30.269290
b_bar(1.000, 262.263) = 30.870508

```

Step 4: Visualize the projected local volatility surface

After fitting the projected local volatility function $\bar{b}(t, s)$, it is useful to inspect it visually as a surface over time and basket value.

At this stage, we have constructed a deterministic function

$$(t, s) \mapsto \bar{b}(t, s),$$

which approximates the local volatility of the one-dimensional projected process.

The goal of this step is to evaluate $\bar{b}(t, s)$ on a grid in the (t, s) -plane and display the result as a 3D wireframe surface.

0.5.11 Function used: `plot_local_volatility`

Syntax

```

fig, ax = plot_local_volatility(
    b_bar, basket0, t,
    K=50, L=150,
    title='Local Volatility Surface',
    color='C0',
    save_path=None
)

```

What it does - Uses the pilot basket paths `basket0` to determine a realistic basket domain at each time through empirical percentiles. - Builds a time-space grid (T_{ij}, S_{ij}) . - Evaluates the fitted local volatility on that grid:

$$B_{ij} = \bar{b}(T_{ij}, S_{ij}).$$

- Produces a 3D wireframe plot of the resulting surface. - Returns the Matplotlib figure and axis objects.

0.5.12 Why the pilot basket paths are used here

The surface is not plotted over an arbitrary rectangle.

Instead, the function uses the pilot sample `basket0` from Step 1 to define time-dependent lower and upper basket bounds.

This is important because the regression is most reliable on the region actually visited by the basket process.

Using basket percentiles avoids focusing the plot on unrealistic extrapolation zones.

0.5.13 Mathematical role of this step

This step does not change the fitted model.

It only evaluates and displays the function already obtained in Step 3.

So the logic is: - Step 3 produces the fitted object $\bar{b}(t, s)$, - Step 4 displays its geometry over the relevant domain.

This gives the first practical verification that the Legendre fit behaves like a reasonable local volatility surface.

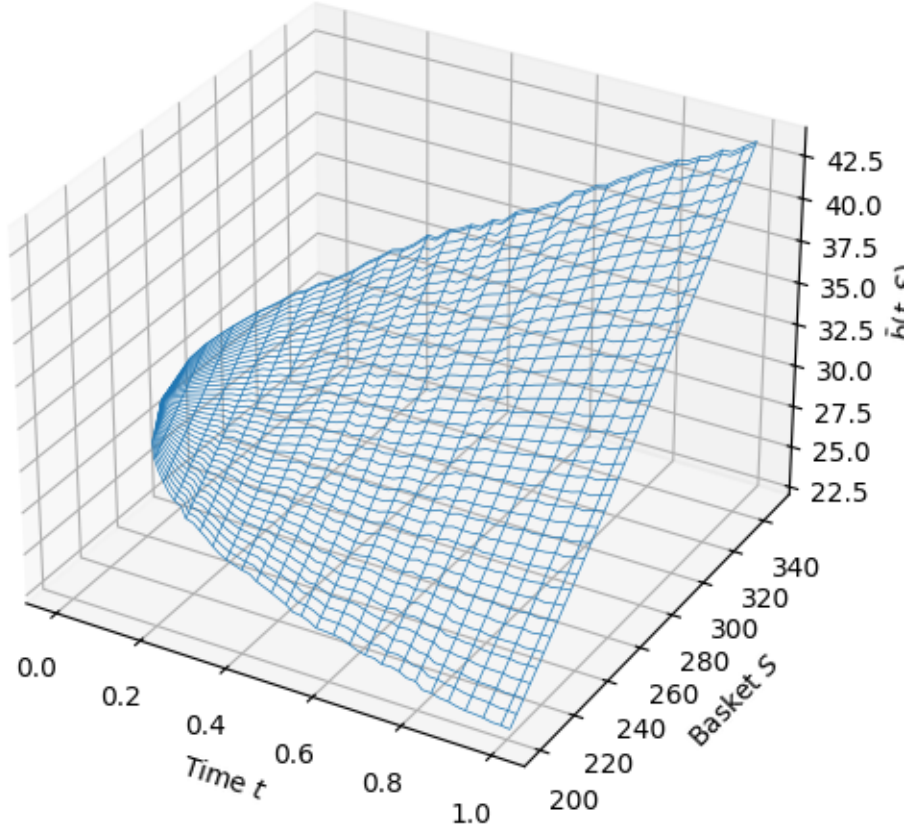
```
[16]: show_full_function(plot_local_volatility)
```

<IPython.core.display.HTML object>

```
[16]: # Step 4: visualize the projected local volatility surface

fig, ax = plot_local_volatility(
    b_bar=b_bar,
    basket=basket0,
    t=t,
    K=50,
    L=150,
    title=f'Single Level Volatility Surface (max degree = {max_deg})',
    color='C0',
    save_path='SLmd2.pdf'
)
plt.show()
```

Single Level Volatility Surface (max degree = 2)



Step 5: Validate the Markovian projection through terminal log-return distributions

After fitting the projected local volatility function $\bar{b}(t, s)$, we now check whether the corresponding one-dimensional projected process reproduces the terminal distribution of the true basket.

This is one of the key consequences expected from Markovian projection: the projected process should preserve the one-dimensional marginal law of the basket, even though it does not reproduce the full high-dimensional path structure.

0.5.14 The two processes being compared

The first process is the true basket

$$S_T = P_1^\top X_T,$$

where X_T comes from the correlated d -dimensional Black–Scholes model.

The second process is the projected one-dimensional diffusion

$$d\bar{S}_t = r\bar{S}_t dt + \bar{b}(t, \bar{S}_t) dW_t, \quad \bar{S}_0 = S_0.$$

The purpose of this step is to compare the terminal distributions of S_T and $\bar{S}_T$.

0.5.15 Why log-returns are used

Instead of comparing the terminal values directly, we compare the terminal log-returns

$$\log\left(\frac{S_T}{S_0}\right), \quad \log\left(\frac{\bar{S}_T}{S_0}\right).$$

This is useful because: - log-returns are scale-invariant, - they are standard in stochastic finance, - and they summarize the terminal distribution in a clean and interpretable way.

0.5.16 Function used: `validate_log_returns`

Syntax

```
stats = validate_log_returns(  
    b_bar, x0, P1, vol, cov_mat, r, dt, N_t,  
    M=10000,  
    save_path=None,  
    show_plot=True  
)
```

What it does - Simulates the projected one-dimensional process using the fitted function `b_bar`. - Simulates the true correlated basket process. - Computes the terminal log-returns of both models. - Compares their empirical means and standard deviations. - Produces an overlaid histogram of the two return distributions. - Returns a dictionary containing summary statistics and errors.

0.5.17 What is being checked mathematically

If the Markovian projection is accurate, then the distribution of $\bar{S}_T$ should be close to the distribution of S_T .

So this step tests whether

$$\mathcal{L}(\log(S_T/S_0)) \approx \mathcal{L}(\log(\bar{S}_T/S_0)),$$

where $\mathcal{L}$ denotes the probability law.

This is not a pathwise comparison.

Rather, it is a comparison of the terminal marginal distributions, which is exactly the relevant notion for Gyöngy-type Markovian projection.

0.5.18 Why this step matters

This is the first direct probabilistic validation of the fitted Legendre model.

- Step 3 gave the local volatility function,
- Step 4 gave a geometric visualization,
- Step 5 now checks whether the projected process actually reproduces the target terminal law.

So this step verifies that the fitted function $\bar{b}(t, s)$ is not only smooth and numerically stable, but also distributionally meaningful.

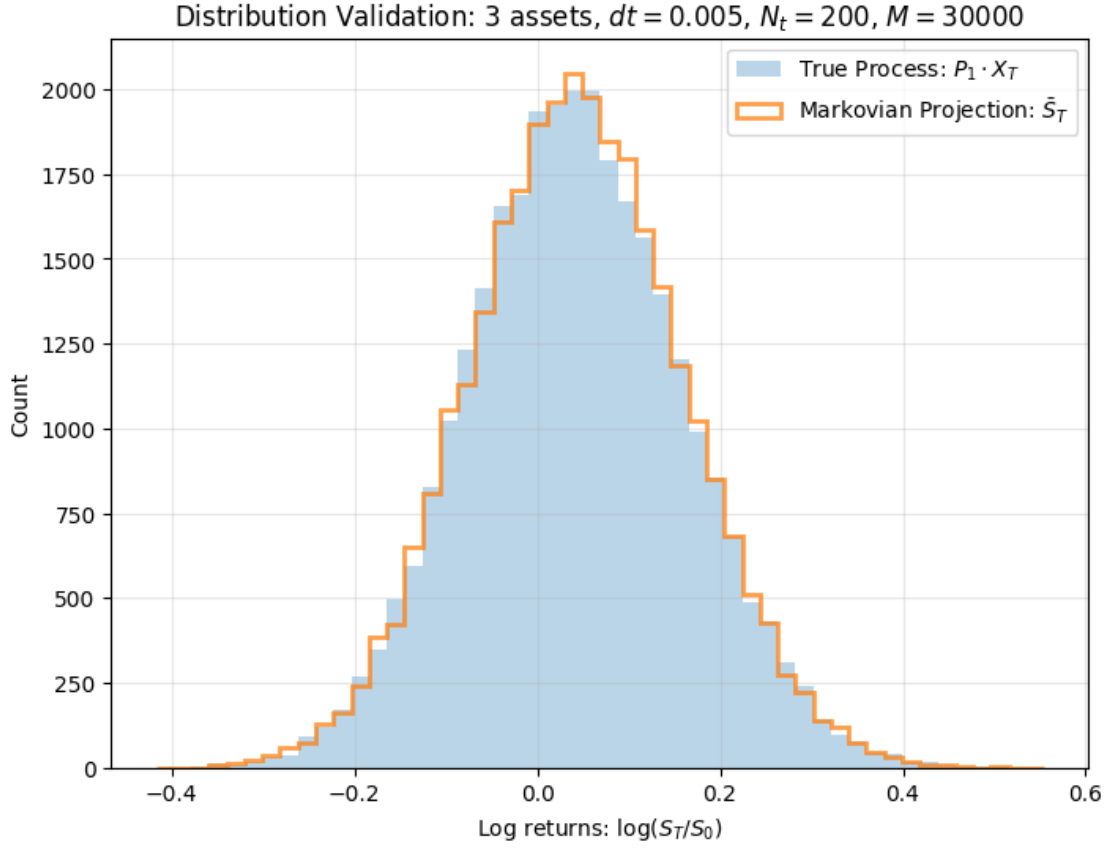
```
[29]: show_full_function(validate_log_returns)
```

<IPython.core.display.HTML object>

[17]: *# Step 5: validate terminal log-return distributions*

```
stats = validate_log_returns(  
    b_bar=b_bar,  
    x0=x0,  
    P1=P1,  
    vol=vol,  
    cov_mat=cov_mat,  
    r=r,  
    dt=dt,  
    N_t=N_t,  
    M=30000,  
    save_path=None,  
    show_plot=True  
)  
  
print("\n--- Step 5 summary ---")  
for key, value in stats.items():  
    print(f"{key}: {value}")
```

True process: mean = 0.042379, std = 0.118111
Projected process: mean = 0.043067, std = 0.117142
Absolute errors: Δ mean = 0.000687, Δ std = 0.000968
Relative errors: Δ mean = 1.62%, Δ std = 0.82%



--- Step 5 summary ---

```

mean_true: 0.04237947151276517
std_true: 0.11811056346816792
mean_proj: 0.04306650654620103
std_proj: 0.11714237444739352
mean_error: 0.0006870350334358608
std_error: 0.0009681890207744043

```

Step 6: Weak error analysis and convergence study

In this final step, we study the quality of the Markovian projection from the point of view of **option pricing**.

The key quantity is the weak error

$$\varepsilon_{\text{weak}} = \frac{|\mathbb{E}[g(S_T)] - \mathbb{E}[g(\bar{S}_T)]|}{|\mathbb{E}[g(S_T)]|},$$

where: - $S_T = P_1^\top X_T$ is the terminal basket value of the true model, - $\bar{S}_T$ is the terminal value of the projected one-dimensional process, - g is a payoff function.

In this implementation, the payoff is the at-the-money European call

$$g(x) = (x - K)^+, \quad K = S_0 = P_1^\top x_0.$$

So the weak error becomes

$$\varepsilon_{\text{weak}} = \frac{|\mathbb{E}[(S_T - K)^+] - \mathbb{E}[\bar{S}_T - K)^+]|}{|\mathbb{E}[(S_T - K)^+]|}.$$

0.5.19 Purpose of this step

The distribution validation in Step 5 checks whether the projected process reproduces the basket law at maturity.

The weak error analysis goes one step further: it checks whether the projected process reproduces **prices of functionals** of the terminal state.

This is the relevant notion for derivative pricing.

0.5.20 Function used: `compute_weak_error`

Syntax

```
weak_errors = compute_weak_error(
    b_bar, x0, P1, vol, cov_mat, r, dt, N_t, M_samples, trials=10
)
```

What it does - Simulates the projected one-dimensional process for each sample size in `M_samples`.
 - Simulates the true basket process for the same sample sizes. - Computes the Monte Carlo call price under both models. - Returns the relative weak errors for all sample sizes and all trials.

So for each sample size M , the function computes several independent weak-error estimates, which allows us to estimate not only the mean weak error but also its variability.

0.5.21 Why several polynomial degrees are compared

The file `SL_weak_error_analysis.py` is not only about Monte Carlo sample size. It also compares different Legendre degrees

$$d_{\max} \in \{3, 2, 1, 0\}.$$

This allows us to study the trade-off between: - **regression error**, which depends on the basis richness, - **Monte Carlo error**, which decreases as the number of validation paths grows.

In rough terms,

$$\varepsilon_{\text{weak}}(M) \approx \varepsilon_{\text{reg}} + CM^{-1/2}.$$

So: - increasing M reduces the Monte Carlo component, - increasing the polynomial degree typically reduces the regression component.

0.5.22 Function used: `plot_convergence_study`

Syntax

```
fig, ax = plot_convergence_study(  
    M_samples, weak_errors_dict, max_degrees,  
    save_path=None, show_plot=True  
)
```

What it does - Takes the weak-error results for several polynomial degrees. - Computes mean and standard deviation across trials. - Produces a log-log convergence plot. - Displays the empirical weak-error decay and compares it with the reference slope

$$M^{-1/2}.$$

This plot separates visually: - Monte Carlo convergence, - and the error floor induced by the regression approximation.

0.5.23 Function used: `export_results_csv`

Syntax

```
export_results_csv(M_samples, weak_errors_dict, max_degrees, filename="weak_error_results.csv")
```

What it does - Saves all weak-error values to a CSV file. - This is useful for later analysis, post-processing, or inclusion in reports.

0.5.24 Mathematical role of this step

This final step is the strongest numerical validation in the notebook.

- Step 4 checked the geometry of the fitted local volatility.
- Step 5 checked terminal distribution matching.
- Step 6 checks whether the projected model is accurate for **pricing expectations**.

So this is the most financially meaningful validation of the Legendre single-level projection.

```
[31]: show_full_function(compute_weak_error)
```

<IPython.core.display.HTML object>

```
[32]: show_full_function(plot_convergence_study)
```

<IPython.core.display.HTML object>

```
[33]: show_full_function(export_results_csv)
```

<IPython.core.display.HTML object>

```
[18]: # Step 6: weak error analysis across polynomial degrees  
  
max_deg_test = [3, 2, 1, 0]  
M_samples = [2000, 4000, 8000, 16000, 32000, 64000, 100000]
```

```

trials = 10

weak_errors_dict = {}

for deg in max_deg_test:
    print(f"\nProcessing polynomial degree = {deg}")

    # Basis for current degree
    pairs_deg = tot_degree_poly(deg)

    # Fit projected volatility for current degree
    D_deg, psi_deg = normaleq_components_SL(
        paths=paths,
        P1=P1,
        pairs=pairs_deg,
        cov_mat=cov_mat,
        vol=vol,
        s_min=s_min0,
        s_max=s_max0,
        T=T
    )

    c_deg = fit_local_vol(D_deg, psi_deg)
    b_bar_deg = make_b_bar(c_deg, pairs_deg, s_min0, s_max0, T, deg)

    # Weak error computation
    weak_errors = compute_weak_error(
        b_bar=b_bar_deg,
        x0=x0,
        P1=P1,
        vol=vol,
        cov_mat=cov_mat,
        r=r,
        dt=dt,
        N_t=N_t,
        M_samples=M_samples,
        trials=trials
    )

    weak_errors_dict[deg] = weak_errors

# Plot convergence study
fig, ax = plot_convergence_study(
    M_samples=M_samples,
    weak_errors_dict=weak_errors_dict,
    max_degrees=max_deg_test,
    save_path='weakSLmd02.pdf',

```

```

    show_plot=True
)

# Optional export
export_results_csv(
    M_samples=M_samples,
    weak_errors_dict=weak_errors_dict,
    max_degrees=max_deg_test,
    filename="weak_error_results.csv"
)

```

Processing polynomial degree = 3

```

Processing M = 2000 with 10 trials... mean = 0.0356, std = 0.0223
Processing M = 4000 with 10 trials... mean = 0.0223, std = 0.0158
Processing M = 8000 with 10 trials... mean = 0.0141, std = 0.0108
Processing M = 16000 with 10 trials... mean = 0.0074, std = 0.0037
Processing M = 32000 with 10 trials... mean = 0.0070, std = 0.0035
Processing M = 64000 with 10 trials... mean = 0.0021, std = 0.0027
Processing M = 100000 with 10 trials... mean = 0.0048, std = 0.0037

```

Processing polynomial degree = 2

```

Processing M = 2000 with 10 trials... mean = 0.0301, std = 0.0209
Processing M = 4000 with 10 trials... mean = 0.0221, std = 0.0143
Processing M = 8000 with 10 trials... mean = 0.0124, std = 0.0113
Processing M = 16000 with 10 trials... mean = 0.0087, std = 0.0070
Processing M = 32000 with 10 trials... mean = 0.0107, std = 0.0080
Processing M = 64000 with 10 trials... mean = 0.0076, std = 0.0038
Processing M = 100000 with 10 trials... mean = 0.0026, std = 0.0032

```

Processing polynomial degree = 1

```

Processing M = 2000 with 10 trials... mean = 0.0413, std = 0.0263
Processing M = 4000 with 10 trials... mean = 0.0135, std = 0.0140
Processing M = 8000 with 10 trials... mean = 0.0223, std = 0.0147
Processing M = 16000 with 10 trials... mean = 0.0135, std = 0.0079
Processing M = 32000 with 10 trials... mean = 0.0082, std = 0.0052
Processing M = 64000 with 10 trials... mean = 0.0032, std = 0.0021
Processing M = 100000 with 10 trials... mean = 0.0045, std = 0.0040

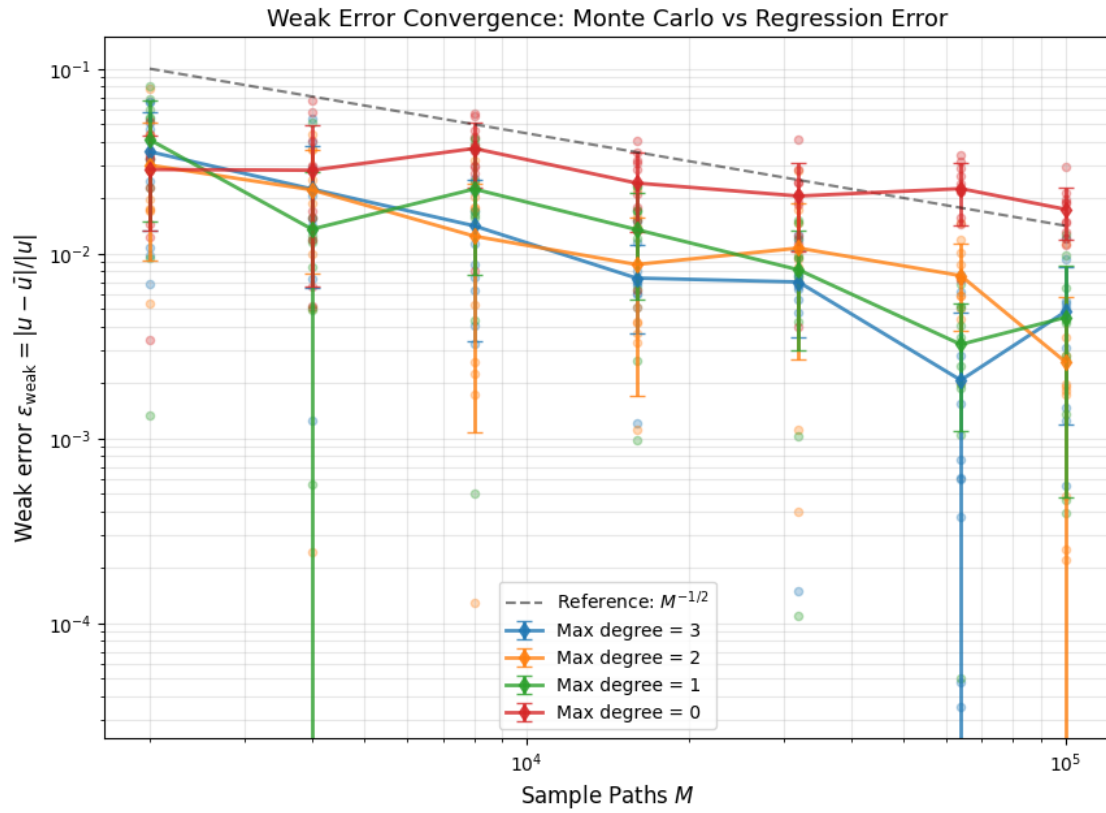
```

Processing polynomial degree = 0

```

Processing M = 2000 with 10 trials... mean = 0.0285, std = 0.0151
Processing M = 4000 with 10 trials... mean = 0.0282, std = 0.0215
Processing M = 8000 with 10 trials... mean = 0.0370, std = 0.0145
Processing M = 16000 with 10 trials... mean = 0.0241, std = 0.0109
Processing M = 32000 with 10 trials... mean = 0.0205, std = 0.0103
Processing M = 64000 with 10 trials... mean = 0.0224, std = 0.0082
Processing M = 100000 with 10 trials... mean = 0.0174, std = 0.0055

```



Results exported to weak_error_results.csv

[]:

nb05_Legendre_Multi_Level

April 8, 2026

```
[1]: # --- Setup: base imports ---
import sys
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import inspect
import html
from IPython.display import display, HTML

# --- Setup: project path ---
PROJECT_ROOT = Path.cwd().parent
if str(PROJECT_ROOT) not in sys.path:
    sys.path.append(str(PROJECT_ROOT))

# Add Original_Programs folder itself because some files use sibling imports
ORIGINAL_ML_DIR = PROJECT_ROOT / "Legendre_Polynomials" / "Multi_Level" / "Original_Programs"
if str(ORIGINAL_ML_DIR) not in sys.path:
    sys.path.append(str(ORIGINAL_ML_DIR))

SINGLE_LEVEL_DIR = PROJECT_ROOT / "Legendre_Polynomials" / "Single_Level"
if str(SINGLE_LEVEL_DIR) not in sys.path:
    sys.path.append(str(SINGLE_LEVEL_DIR))

# --- Optional (recommended) for syntax highlighting ---
from pygments import highlight
from pygments.lexers import PythonLexer
from pygments.formatters import HtmlFormatter

_PYGMENTS_CSS_LOADED = False

def show_full_function(func, name=None):
    global _PYGMENTS_CSS_LOADED

    name = name or func.__name__
```

```

source = inspect.getsource(func)

formatter = HtmlFormatter(style="friendly", noclasses=False)
highlighted = highlight(source, PythonLexer(), formatter)

if not _PYGMENTS_CSS_LOADED:
    css = formatter.get_style_defs('.highlight')
    display(HTML(f"<style>{css}</style>"))
    _PYGMENTS_CSS_LOADED = True

html_block = f"""
<details>
  <summary><strong>View full function: {html.escape(name)}</strong></
↪summary>
  {highlighted}
</details>
"""
display(HTML(html_block))

# --- Core ML utilities ---
from Legendre_Polynomials.Multi_Level.Original_Programs.ML_level_utilities ↵
↪import (
    GBM_paths,
    tot_degree_poly,
    normaleq_components_ML,
    fit_local_vol,
    make_b_bar,
)

# --- Standard telescoping-sum MLMC ---
from Legendre_Polynomials.Multi_Level.Original_Programs.ML_telescoping_sum ↵
↪import (
    scalings_l0,
    mlmc_l,
    make_c,
)

# --- Weak error analysis ---
from Legendre_Polynomials.Multi_Level.Original_Programs.ML_weak_error_analysis ↵
↪import (
    compute_weak_error_ML,
    plot_weak_error_convergence,
)

from Legendre_Polynomials.Single_Level.SL_surface_visualisation import ↵
↪plot_local_volatility

```

What this Notebook will do

This notebook integrates the whole `Multi_Level/Original_Programs` workflow of the `Legendre` folder:

1. build the Legendre regression model from `ML_level_utilities.py`,
2. Construct the standar telescopic sum,
3. visualize the local volatility surface,
4. study weak error convergence for option pricing.

Multi-Level Markovian Projection with Legendre Polynomial Regression

0.1 Black-Scholes Correlated Model:

We consider a d -dimensional correlated Black–Scholes model

$$dX(t) = a(t, X(t)) dt + b(t, X(t)) dW(t), \quad 0 \leq t \leq T, \quad X(t) \in \mathbb{R}^d, \quad (1)$$

$$X(0) = x_0. \quad (2)$$

where - $W(t)$ is a k -dim. Brownian motion. - $a(t, x) = r x$, $r = 0.05$.
- $b(t, x) \in \mathbb{R}^{d \times k}$.
- $(b_{BS}(t, x))_{ij} = x_i \Sigma_{ij}$.

0.1.1 Example ($d = 3, k = 3$)

Let $x = (x_1, x_2, x_3)^\top$ and $\sigma = (\sigma_1, \sigma_2, \sigma_3)^\top$. Assume the correlated Black–Scholes diffusion is defined by

$$b_{BS}(t, x) = \text{diag}(x) \text{diag}(\sigma) G, \quad \text{equivalently } (b_{BS}(t, x))_{ij} = \sigma_i x_i G_{ij}$$

where,

$$\Sigma_{ij} = \sigma_i G_{ij}, \quad GG^\top = \begin{pmatrix} 1 & 0.8 & 0.3 \\ 0.8 & 1 & 0.1 \\ 0.3 & 0.1 & 1 \end{pmatrix}, \quad \sigma = (0.2, 0.15, 0.1)^\top.$$

Each component satisfies

$$dX_i(t) = rX_i(t) dt + \sigma_i X_i(t) \sum_{j=1}^k G_{ij} dW_j(t), \quad i = 1, \dots, d.$$

Thus, even if the Brownian motions $W_1, \dots, W_k$ are independent, the assets are correlated through the matrix G .

0.2 Markovian Projection

Let $S(t) = P_1 X(t)$, with $P_1 \in \mathbb{R}^{1 \times d}$ and $X(t) \in \mathbb{R}^{d \times 1}$. Define the Markovian surrogate $\bar{S}^{(x_0)}(t)$ by

$$\begin{cases} d\bar{S}^{(x_0)}(t) = \bar{a}^{(x_0)}(t, \bar{S}^{(x_0)}(t)) dt + \bar{b}^{(x_0)}(t, \bar{S}^{(x_0)}(t)) d\bar{W}(t), & t \in [0, T], \\ \bar{S}^{(x_0)}(0) = P_1 x_0. \end{cases}$$

The drift and diffusion coefficients are given by conditional expectations:

$$\bar{a}^{(x_0)}(t, s) = \mathbb{E}[P_1 a(t, X(t)) \mid P_1 X(t) = s, X(0) = x_0] = \mathbb{E}[r P_1 X(t) \mid P_1 X(t) = s, X(0) = x_0] = r s,$$

and

$$(\bar{b}^{(x_0)}(t, s))^2 = \mathbb{E} \left[P_1 b_{\text{BS}}(t, X(t)) b_{\text{BS}}(t, X(t))^\top P_1^\top \mid P_1 X(t) = s, X(0) = x_0 \right].$$

For $P_1 = \frac{1}{d}(1, \dots, 1)$ and $C = GG^\top$, this becomes

$$(\bar{b}^{x_0}(t, s))^2 = \mathbb{E} \left[\frac{1}{d^2} \sum_{i=1}^d \sum_{j=1}^d \sigma_i \sigma_j X_i(t) X_j(t) C_{ij} \mid P_1 X(t) = s, X(0) = x_0 \right],$$

0.3 L^2 Regression Problem

We approximate $(\bar{b}^{x_0})^2(\cdot, \cdot)$ by solving the time-integrated L^2 problem

$$(\bar{b}^{x_0})^2(\cdot, \cdot) \approx \arg \min_{h \in V} \int_0^T \mathbb{E} \left[\left(P b_{\text{BS}}(t, X(t)) b_{\text{BS}}(t, X(t))^\top P^\top - h(t, PX(t)) \right)^2 \mid X(0) = x_0 \right] dt.$$

For example, one may choose a parametric class

$$V = \{ h : h(t, x) = c_0 + c_1 t + c_2 x \}.$$

Discretizing time with a grid $0 = t_0 < t_1 < \dots < t_N = T$, an empirical Monte Carlo objective is

$$(\bar{b}^{x_0})^2(\cdot, \cdot) \approx \arg \min_{h \in V} \frac{1}{M} \sum_{m=1}^M \frac{1}{N} \sum_{n=1}^N \left(\frac{1}{d^2} \sum_{i=1}^d \sum_{j=1}^d \sigma_i \sigma_j X_i^{(m)}(t_n) X_j^{(m)}(t_n) C_{ij} - h(t_n, PX^{(m)}(t_n)) \right)^2.$$

Ansatz. We approximate the squared projected diffusion by a finite expansion

$$\tilde{b}^2(t, s) \approx \sum_{p=1}^P c_p \psi_p(t, s).$$

Discrete L^2 Projection

Let $y \in \Gamma \subset \mathbb{R}^N$ be a random parameter with density ρ , and let

$$u : \Gamma \rightarrow V$$

be the solution map, where V is a Hilbert space. For each realization y , one solves a deterministic problem, so $u(y)$ is the corresponding solution. Since y is random, the natural error measure is the mean-square norm

$$\|u\|_{L^2_\rho(\Gamma; V)}^2 = \int_\Gamma \|u(y)\|_V^2 \rho(y) dy.$$

0.3.1 Discrete L^2 projection (Random PDE context)

We choose a finite-dimensional polynomial space with basis $\{\psi_p\}_{p \in \Lambda}$, where Λ is a finite index set. Then we approximate u in the space

$$\mathbb{P}_\Lambda(\Gamma) \otimes V = \left\{ v(y) = \sum_{p \in \Lambda} \hat{v}_p \psi_p(y), \quad \hat{v}_p \in V \right\}.$$

Here, the coefficients $\hat{v}_p$ belong to V , while the dependence on y is described by polynomials. The best approximation is the L^2_ρ -projection of u onto this space:

$$\Pi_\Lambda u = \arg \min_{v \in \mathbb{P}_\Lambda(\Gamma) \otimes V} \|u - v\|_{L^2_\rho(\Gamma; V)}^2.$$

This means that $\Pi_\Lambda u$ is the best polynomial approximation of u in the mean-square sense.

If the basis is orthonormal, then

$$\Pi_\Lambda u(y) = \sum_{p \in \Lambda} \hat{u}_p \psi_p(y), \quad \hat{u}_p = \mathbb{E}[u(y) \psi_p(y)].$$

0.3.2 Transition to the discrete formulation

The previous projection is well defined, but in practice it cannot usually be computed exactly, because it involves expectations such as

$$\mathbb{E}[u(y) \psi_p(y)].$$

In applications, $u(y)$ is not known in closed form. Instead, one evaluates it at sampled points

$$y^{(1)}, \dots, y^{(M)},$$

and obtains the corresponding snapshots

$$u^{(k)} = u(y^{(k)}), \quad k = 1, \dots, M.$$

This leads to a computable approximation of the L^2 -projection.

0.3.3 L^2 discrete projection

We replace the exact L^2_ρ -norm by the empirical norm

$$\|g\|_M^2 = \frac{1}{M} \sum_{k=1}^M \|g(y^{(k)})\|_V^2.$$

Then the discrete projection is defined by

$$\widehat{\Pi}_{\Lambda, M} u = \arg \min_{v \in \mathbb{P}_{\Lambda}(\Gamma) \otimes V} \frac{1}{M} \sum_{k=1}^M \|u^{(k)} - v(y^{(k)})\|_V^2.$$

Thus, the exact projection

$$\Pi_{\Lambda} u = \arg \min \|u - v\|_{L^2_{\rho}(\Gamma; V)}^2$$

is replaced by the computable discrete projection

$$\widehat{\Pi}_{\Lambda, M} u = \arg \min \|u - v\|_M^2.$$

0.3.4 Algebraic form

Write the approximation as

$$v(y) = \sum_{p \in \Lambda} c_p \psi_p(y).$$

Evaluating at the sample points gives

$$v(y^{(k)}) = \sum_{p \in \Lambda} c_p \psi_p(y^{(k)}).$$

Define the design matrix D by

$$D_{k,p} = \psi_p(y^{(k)}), \quad k = 1, \dots, M, \quad p \in \Lambda.$$

Then the coefficients $c = (c_p)_{p \in \Lambda}$ are obtained from the least-squares problem, which leads to the normal equations

$$D^T D c = D^T U,$$

where U collects the sampled values $u^{(k)}$.

Therefore, the discrete L^2 -projection becomes a standard least-squares regression problem.

Multi-Level discrete L^2 Projection Assume that the target quantity depends on a parameter $y \in \Gamma$, and let

$$f_{n_{\ell}} : \Gamma \rightarrow \mathbb{R}, \quad \ell = 0, 1, \dots, L,$$

be a hierarchy of numerical approximations of a target function f . Here, the index n_{ℓ} refers to the **solver discretization level**, for example a time step or mesh refinement. Thus, for each fixed y , the quantity $f_{n_{\ell}}(y)$ is the approximation produced by the level- ℓ solver.

We assume that

$$\|f - f_{n_\ell}\|_{L^2_\mu(\Gamma)} \rightarrow 0 \quad \text{as } \ell \rightarrow \infty,$$

that is, the numerical approximations converge to f in mean square over the parameter space Γ .

The key identity is the telescoping decomposition

$$f_{n_L} = f_0 + \sum_{\ell=1}^L (f_{n_\ell} - f_{n_{\ell-1}}).$$

Instead of approximating f_L directly by a single least-squares fit, the multilevel idea is to approximate each correction separately.

Similarly, we introduce

$$\Lambda_{m_0} \subset \Lambda_{m_1} \subset \dots \subset \Lambda_{m_k} \dots$$

such that

$$\lim_{k \rightarrow \infty} \inf_{v \in V_{\Lambda_{m_k}}} \|f - v\|_{L^2_\mu} = 0$$

For each Λ_{m_k} we introduce a weighted discrete least square projection operator $\widehat{\Pi}_{M_k}$ using $O(|\Lambda_{m_k}|)$ random points. Given a maximum level $L \in \mathbb{N}$:

$$S_L(f) = \widehat{\Pi}_L(f_0) + \sum_{\ell=1}^L \widehat{\Pi}_{L-\ell}(f_\ell - f_{\ell-1}).$$

L^2 Projection SDES

Recall:

$$(\bar{b}^{x_0})^2 \approx \arg \min_{h \in V} \frac{1}{M} \sum_m \frac{1}{N} \sum_n (P_1^T b(t_n, X^m(t_n)) b(t_n, X^m(t_n))^T P_1 - h(t_n, P_1 X^m(t_n)))^2$$

where X^m are independent realizations of X . Denote

$$g := P_1^T b(t, X(t)) b(t, X(t))^T P_1.$$

$$S_L(f) := \Pi_{V_L} g_0(X_0) + \sum_{\ell=1}^L \Pi_{V_{L-\ell}} (g(X^\ell) - g(X^{\ell-1}))$$

where

$$\Pi_{V_{L-\ell}} (g(X^\ell) - g(X^{\ell-1})) = \arg \min_{v \in V_{L-\ell}} \|(g(X^\ell) - g(X^{\ell-1})) - v(\cdot)\|.$$

Compute the projection of correction $\Pi_{V_{L-\ell}}(g(X^\ell) - g(X^{\ell-1}))$ by

$$\arg \min_{h \in V} \frac{1}{M_\ell} \sum_{m=1}^{M_\ell} \frac{1}{N_{\ell-1}} \sum_{n=1}^{\mathbf{N}_{-1}} \left(\left[P^\top b(t_n, X_\ell^m(t_n^{\ell-1})) b(t_n, X_\ell^m(t_n^{\ell-1}))^\top P \right. \right. \\ \left. \left. - P^\top b(t_n, X_{\ell-1}^m(t_n^{\ell-1})) b(t_n, X_{\ell-1}^m(t_n^{\ell-1}))^\top P \right] - h(t_n^{\ell-1}, P X_\ell^m(t_n^{\ell-1})) \right)^2 \quad (3)$$

$$D \in \mathbb{R}^{M_\ell N_{\ell-1} \times |\Lambda|}, \Psi \in \mathbb{R}^{M_\ell N_{\ell-1}}, c_\ell \in \mathbb{R}^{|\Lambda_{L-\ell}|} \quad (4)$$

$$M_\ell = C \times \dim(V_{L-\ell})^2, \quad V_{L-\ell} = \text{span} \left\{ \tilde{P}_i(\tau(t)) \tilde{P}_j(\xi(s)) : i + j \leq L - \ell \right\}. \quad (5)$$

The estimated coefficient is

$$(\bar{b}^{x_0})^2(t, s) \approx \sum_{p \in \Lambda_L} c_p \psi(t, s), \quad c = \{c_p\}_{p \in \Lambda_L} = \sum_{\ell=0}^L c_\ell \quad (6)$$

- Assume that there exists $\theta_{\Delta t} : X^\ell \rightarrow X^{\ell-1}$ such that $X^{\ell-1} \rightarrow X^\ell$ as $\Delta t \rightarrow 0$.

$$\Rightarrow \Pi_{V_{L-\ell}}(g(X^\ell) - g(X^{\ell-1})) := \arg \min_{v \in V_{L-\ell}} \|(g(X^\ell) - g(\theta_{\Delta t}(X^\ell))) - v(X^\ell)\|.$$

This map will depend on the particular path in question: $\theta_{\Delta t}(X^\ell, \omega)$.

- Consider a sequence of exponentially increasing polynomial subspaces $(V_\ell)_{\ell=0}^\infty$: we choose subspaces of (total degree) Legendre polynomials which are orthogonal on $[-1, 1]$ with weight function $w = 1$.
- Exponentially decreasing time steps $h_0, h_1, \dots, h_\ell, \dots$: we choose

$$h_\ell \approx h_0 2^{-\ell}.$$

Implementation

Implementation — Step 1: Define the multi-level setting and estimate the basket scaling domain

In this first implementation step, we initialize the market parameters, define the multi-level hierarchy, and estimate the basket-value interval used for Legendre scaling.

The stochastic model is the same correlated Black–Scholes basket:

$$dX_i(t) = rX_i(t) dt + \sigma_i X_i(t) (G dW(t))_i, \quad i = 1, \dots, d,$$

with equal-weight basket

$$S(t) = P_1^\top X(t), \quad P_1 = \frac{1}{d}(1, \dots, 1)^\top.$$

The new ingredient is the **multi-level hierarchy of timesteps**

$$h_l = h_0 2^{-l}, \quad l = 0, 1, \dots, L,$$

where: - h_0 is the coarsest timestep, - L is the finest level.

In this implementation, the maximum polynomial degree and the number of levels are tied together: the finest level corresponds to `max_deg`, and this determines the pilot discretization used to estimate the basket domain.

Before building the level-wise regression systems, we need a stable spatial interval

$$[s_{\min}, s_{\max}]$$

for the basket variable. This is required because the Legendre basis will later be evaluated on the scaled interval $[-1, 1]$ through

$$\xi(s) = \frac{2(s - s_{\min})}{s_{\max} - s_{\min}} - 1.$$

0.3.5 Function used: `GBM_paths`

Syntax

```
paths = GBM_paths(x0, r, vol, cov_mat, dt, N_t, M_t)
```

What it does - Simulates correlated d -dimensional geometric Brownian motion paths. - Uses the Cholesky factor of the correlation matrix to generate correlated Brownian increments. - Returns the path array used for Monte Carlo training or pilot estimation.

In this step, `GBM_paths` is not called directly in the notebook cell, but it is used internally by `scalings_10(...)`.

0.3.6 Function used: `scalings_10`

Syntax

```
s_min0, s_max0, basket0 = scalings_10(
    x0, T, h0, r, cov_mat, vol, max_deg, P1,
    M_0=10000, return_basket=True
)
```

What it does - Runs a pilot simulation at the **finest timestep**

$$h_{\text{fine}} = h_0 2^{-L}, \quad L = \text{max_deg},$$

so that the basket range is estimated on the most resolved grid. - Computes empirical lower and upper percentile bounds of the basket values. - Returns: - `s_min0`: lower basket bound, - `s_max0`: upper basket bound, - `basket0`: pilot basket paths.

These bounds define the spatial interval on which the Legendre approximation will be fitted.

0.3.7 Mathematical role of this step

This step prepares the multi-level construction.

It fixes: - the basket model, - the coarsest time resolution h_0 , - the finest level L , - and the basket scaling interval.

So before any telescoping correction is computed, we already have the global domain on which the level-wise Legendre approximations will be built.

```
[2]: # Step 1: define the multi-level setting and estimate the scaling domain
```

```
# Basket configuration
d = 3
P1 = np.ones(d) / d

# Market parameters
r = 0.05
x0 = np.linspace(225, 275, num=d)[: , np.newaxis]
vol = np.array([0.2, 0.15, 0.1])
cov_mat = np.array([
    [1.0, 0.8, 0.3],
    [0.8, 1.0, 0.1],
    [0.3, 0.1, 1.0]
])

# Time horizon and multi-level hierarchy
T = 1.0
h0 = 0.125          # coarsest timestep
max_deg = 3         # also determines finest level L = max_deg

# Pilot run for basket scaling domain
s_min0, s_max0, basket0 = scalings_l0(
    x0=x0,
    T=T,
    h0=h0,
    r=r,
    cov_mat=cov_mat,
    vol=vol,
    max_deg=max_deg,
    P1=P1,
    M_0=10000,
    return_basket=True
)

print("\n--- Step 1 summary ---")
print("d =", d)
print("P1 =", P1)
print("T =", T)
print("h0 =", h0)
print("max_deg =", max_deg)
print("number of ML levels =", max_deg + 1)
print("finest timestep =", h0 * 2**(-max_deg))
print("s_min0 =", s_min0)
print("s_max0 =", s_max0)
print("basket0 shape =", basket0.shape)
```

```

--- Step 1 summary ---
d = 3
P1 = [0.33333333 0.33333333 0.33333333]
T = 1.0
h0 = 0.125
max_deg = 3
number of ML levels = 4
finest timestep = 0.015625
s_min0 = 183.22550236906832
s_max0 = 383.03378285854853
basket0 shape = (10000, 64)

```

```
[20]: show_full_function(GBM_paths)
```

<IPython.core.display.HTML object>

```
[21]: show_full_function(scalings_l0)
```

<IPython.core.display.HTML object>

Implementation — Step 2: Define the level-dependent polynomial spaces

Before constructing any multi-level correction, we must understand how the approximation space changes from level to level.

In the standard multi-level implementation, the time discretization becomes finer as the level increases:

$$h_l = h_0 2^{-l}, \quad l = 0, 1, \dots, L.$$

At the same time, the polynomial approximation space is made **level-dependent**.

If the maximum polynomial degree is `max_deg`, then at level l the implementation uses

$$d_l = \text{max_deg} - l.$$

So: - coarse levels use richer polynomial spaces, - fine levels use smaller correction spaces.

This is natural in the multi-level setting, because finer levels are intended only to correct what is missing from the coarser approximation.

0.3.8 Function used: `tot_degree_poly`

Syntax

```
pairs_l = tot_degree_poly(deg_l)
```

What it does - Generates the set of basis index pairs

$$\Lambda_l = \{(i, j) \in \mathbb{N}_0^2 : i + j \leq d_l\},$$

where d_l is the polynomial degree used at level l . - Each pair (i, j) corresponds to the basis function

$$\widetilde{P}_i(\tau(t)) \widetilde{P}_j(\xi(s)).$$

0.3.9 Why the degree decreases with the level

In the multi-level telescoping construction, level 0 carries the main coarse approximation, while higher levels only provide corrections.

So the implementation uses: - more basis functions on coarse levels, - fewer basis functions on fine levels.

This keeps the cost under control while preserving the telescoping structure of the approximation.

0.3.10 Mathematical role of this step

This step defines the level-wise approximation spaces

$$V_l = \text{span} \left\{ \widetilde{P}_i(\tau(t)) \widetilde{P}_j(\xi(s)) : i + j \leq d_l \right\}.$$

The next step will use these spaces to build the actual level correction estimator.

```
[3]: # Step 2: inspect the level-dependent polynomial spaces

level_info = []

for l in range(max_deg + 1):
    deg_l = max_deg - l
    pairs_l = tot_degree_poly(deg_l)

    level_info.append((l, deg_l, len(pairs_l), pairs_l[:6]))

    print(f"Level l = {l}")
    print(f"  degree d_l = {deg_l}")
    print(f"  number of basis functions = {len(pairs_l)}")
    print(f"  first basis pairs = {pairs_l[:6]}")
    print()
```

```
Level l = 0
  degree d_l = 3
  number of basis functions = 10
  first basis pairs = [(0, 0), (0, 1), (0, 2), (0, 3), (1, 0), (1, 1)]
```

```
Level l = 1
  degree d_l = 2
  number of basis functions = 6
  first basis pairs = [(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (2, 0)]
```

```
Level l = 2
  degree d_l = 1
  number of basis functions = 3
  first basis pairs = [(0, 0), (0, 1), (1, 0)]
```

```
Level l = 3
```

```

degree d_l = 0
number of basis functions = 1
first basis pairs = [(0, 0)]

```

[23]: `show_full_function(tot_degree_poly)`

<IPython.core.display.HTML object>

Implementation — Step 3: Compute a single multi-level correction with `mlmc_l`

This is the core step of the standard multi-level method.

At level l , the goal is not to fit the full projected variance directly.

Instead, we fit a **correction** between a fine approximation and a coarse approximation.

If the fine timestep is

$$h_l = h_0 2^{-l},$$

then the corresponding coarse timestep is

$$h_{l-1} = 2h_l.$$

The multi-level idea is to approximate the level correction

$$\bar{b}_l^2(t, s) - \bar{b}_{l-1}^2(t, s),$$

and then sum these corrections across levels.

0.3.11 Function used: `mlmc_l`

Syntax

```
c_l = mlmc_l(x0, T, h0, l, r, cov_mat, vol, max_deg, P1, s_min0, s_max0, C=80)
```

What it does - Builds the level- l regression correction. - Generates coupled fine and coarse paths. - Constructs the regression system for the volatility difference. - Solves the level-specific least-squares problem. - Returns a padded coefficient vector `c_l` so that all levels can later be summed consistently.

0.3.12 Step 3a: Fine/coarse coupling

The function generates: - fine paths with timestep h_l , - coarse paths with timestep $2h_l$.

The coupling is crucial: both levels use the **same underlying Gaussian randomness**, with the coarse increments obtained by summing consecutive fine increments.

This coupling is what reduces the variance of the level correction.

0.3.13 Step 3b: Level-specific polynomial space

At level l , the code uses the degree

$$d_l = \text{max_deg} - l.$$

So the correction is fitted in the space

$$V_l = \text{span}\left\{\widetilde{P}_i(\tau(t))\widetilde{P}_j(\xi(s)) : i + j \leq d_l\right\}.$$

This means coarse levels carry richer approximation spaces, while finer levels carry smaller correction spaces.

0.3.14 Internal function used: `normaleq_components_ML`

Inside `mlmc_1(...)`, the coupled paths are passed to

`normaleq_components_ML(paths_f, paths_c, P1, pairs, cov_mat, vol, s_min0, s_max0, T)`

This function builds: - the design matrix D_l , - and the target vector ψ_l ,

where the target is now the **difference**

$$\psi_l = b_{\text{fine}}^2 - b_{\text{coarse}}^2.$$

This is the key mathematical difference from the single-level notebook, where the target was only b^2 .

0.3.15 Internal function used: `fit_local_vol`

Once the system

$$D_l c_l \approx \psi_l$$

is built, the coefficients are obtained by QR regression:

`c = fit_local_vol(D, psi)`

So each level produces its own coefficient vector c_l .

0.3.16 Why the coefficients are padded

Different levels use different basis sizes, because the degree decreases with the level.

To make the telescoping sum possible, the coefficient vector from each level is padded to the maximum dimension:

$$c_l \in \mathbb{R}^{P_{\max}}.$$

That way, all level contributions can later be summed componentwise:

$$c_{\text{total}} = \sum_{l=0}^L c_l.$$

0.3.17 Mathematical role of this step

This step computes one single multi-level correction.

So rather than producing the final projected volatility immediately, it produces one building block of the telescoping approximation:

$$c_l \quad \text{for a fixed level } l.$$

The next step will aggregate these level-wise corrections across all levels.

```
[4]: # Step 3: compute one multi-level correction at a representative level

l_demo = 1

c_l_demo = mlmc_l(
    x0=x0,
    T=T,
    h0=h0,
    l=l_demo,
    r=r,
    cov_mat=cov_mat,
    vol=vol,
    max_deg=max_deg,
    P1=P1,
    s_min0=s_min0,
    s_max0=s_max0,
    C=80
)

print("\n--- Step 3 summary ---")
print("demo level l =", l_demo)
print("coefficient vector shape =", c_l_demo.shape)
print("number of nonzero entries =", np.count_nonzero(np.abs(c_l_demo) > 1e-14))
print("first coefficients =", c_l_demo[:10])
```

```
Level 1: polynomial degree = 2
Condition number: 1.29e+01
Smallest 5 singular values: [96.10960351 86.50211072 41.46950407 21.03418398
8.65802745]
```

```
--- Step 3 summary ---
demo level l = 1
coefficient vector shape = (10,)
number of nonzero entries = 6
first coefficients = [21.09355084 20.2455706 15.37920088 -2.86922651
-2.96471593 0.03634017
0. 0. 0. 0. ]
```

```
[25]: show_full_function(mlmc_l)
```

```
<IPython.core.display.HTML object>
```

```
[26]: show_full_function(normaleq_components_ML)
```

```
<IPython.core.display.HTML object>
```

Implementation — Step 4: Aggregate the telescoping corrections and construct the final projected volatility

After Step 3, we know how to compute one level correction coefficient vector (`c_l`) with `mlmc_1(...)`.

Now we combine all levels to obtain the final coefficient vector of the multi-level approximation.

The telescoping construction is:

$$c_{\text{total}} = \sum_{l=0}^L c_l.$$

In the code, this is exactly what `make_c(...)` does.

0.3.18 Function used: `make_c`

Syntax

```
c_total = make_c(x0, T, h0, r, cov_mat, vol, max_deg, P1, s_min0, s_max0, C=80)
```

What it does - Loops over all levels $l = 0, \dots, L$ (with $L = \text{max_deg}$). - Calls `mlmc_1(...)` at each level to compute c_l . - Returns the telescoping sum c_{total} .

So this function is the bridge from level-wise corrections to the final multi-level regression representation.

0.3.19 Function used: `tot_degree_poly`

Syntax

```
pairs = tot_degree_poly(max_deg)
```

What it does - Generates the final basis index set at the maximum degree. - This basis is used to interpret `c_total` and to evaluate the final function.

0.3.20 Function used: `make_b_bar`

Syntax

```
b_bar_ml = make_b_bar(c_total, pairs, s_min0, s_max0, T, max_deg)
```

What it does - Converts the aggregated coefficient vector into a callable projected volatility function. - Scales (t, s) to $[-1, 1]$, evaluates the Legendre expansion, and returns:

$$\bar{b}_{\text{ML}}(t, s) = \sqrt{\tilde{b}_{\text{ML}}^2(t, s)}.$$

0.3.21 Mathematical role of this step

This is the point where the multi-level algorithm produces its final model object.

- Step 3 produced one correction c_l .
- Step 4 sums all corrections into c_{total} .
- Then c_{total} is turned into the final callable volatility $\bar{b}_{\text{ML}}(t, s)$.

From here, we can proceed to validation (weak error, etc.) exactly as we did in the single-level notebook, but now with the multi-level fitted function.

[27]: `show_full_function(make_c)`

<IPython.core.display.HTML object>

```
[28]: show_full_function(make_b_bar)
```

<IPython.core.display.HTML object>

```
[5]: # Step 4: aggregate all ML levels and build final b_bar
```

```
c_total = make_c(  
    x0=x0,  
    T=T,  
    h0=h0,  
    r=r,  
    cov_mat=cov_mat,  
    vol=vol,  
    max_deg=max_deg,  
    P1=P1,  
    s_min0=s_min0,  
    s_max0=s_max0,  
    C=80  
)  
  
pairs_max = tot_degree_poly(max_deg)  
b_bar_ml = make_b_bar(c_total, pairs_max, s_min0, s_max0, T, max_deg)  
  
print("\n--- Step 4 summary ---")  
print("c_total shape =", c_total.shape)  
print("number of basis functions (max level) =", len(pairs_max))  
print("first coefficients =", c_total[:10])  
print("b_bar_ml callable created =", callable(b_bar_ml))  
  
# quick sanity checks  
t_test = np.array([0.0, T/2, T])  
s_test = np.array([  
    float(np.mean(basket0[:, 0])),  
    float(np.mean(basket0[:, basket0.shape[1]//2])),  
    float(np.mean(basket0[:, -1]))  
)  
  
for tt, ss in zip(t_test, s_test):  
    print(f"b_bar_ml({tt:.3f}, {ss:.3f}) = {b_bar_ml(tt, ss):.6f}")
```

Level 0: polynomial degree = 3

Condition number: 8.29e+01

Smallest 5 singular values: [35.51818453 26.46794237 12.80540131 8.27731743
2.71270095]

Level 1: polynomial degree = 2

Condition number: 1.27e+01

```
Smallest 5 singular values: [95.99457092 86.55354159 41.19080038 21.09466595
8.783326 ]
```

```
Level 2: polynomial degree = 1
```

```
Condition number: 3.56e+00
```

```
Smallest 5 singular values: [59.96179826 57.00596736 16.85304738]
```

```
Level 3: polynomial degree = 0
```

```
Condition number: 1.00e+00
```

```
Smallest 5 singular values: [25.29822128]
```

```
--- Step 4 summary ---
```

```
c_total shape = (10,)
```

```
number of basis functions (max level) = 10
```

```
first coefficients = [ 2.53386062e+03  1.19873158e+03  1.64049540e+02
-4.24430036e-01
```

```
-7.11175870e+00  3.49634357e-01  4.86907407e+00 -1.74203841e-01
```

```
2.23760688e-01 -2.85431113e-02]
```

```
b_bar_ml callable created = True
```

```
b_bar_ml(0.000, 250.000) = 29.498847
```

```
b_bar_ml(0.500, 256.404) = 30.284573
```

```
b_bar_ml(1.000, 262.626) = 31.030344
```

Implementation — Step 5: Validate the multi-level fit with weak error convergence

Now we validate the multi-level projected volatility function $\bar{b}_{ML}(t, s)$ through option-pricing weak error.

For payoff

$$g(x) = (x - K)^+, \quad K = S_0 = P_1^\top x_0,$$

the weak error is

$$\varepsilon_{\text{weak}} = \frac{|\mathbb{E}[g(S_T)] - \mathbb{E}[g(\bar{S}_T)]|}{|\mathbb{E}[g(S_T)]|}.$$

Here: - S_T is the terminal basket under the true d -dimensional model, - $\bar{S}_T$ is the terminal value of the projected 1D process driven by $\bar{b}_{ML}$.

0.3.22 Function used: compute_weak_error_ML

Syntax

```
weak_errors = compute_weak_error_ML(
    b_bar_ml, x0, P1, vol, cov_mat, r, dt, N_t, M_samples, trials=10
)
```

What it does - Simulates projected and true models for each sample size $M \in M_{\text{samples}}$. - Computes Monte Carlo estimates of call prices for both models. - Returns trial-wise relative weak errors at each sample size.

So for each M , you get multiple weak-error realizations (one per trial), not just one scalar.

0.3.23 Function used: plot_weak_error_convergence

Syntax

fig, ax = plot_weak_error_convergence(M_samples, weak_errors_dict, max_deg_test, save_path=None)

What it does - Aggregates the weak-error results across trials. - Plots mean + dispersion on log-log axes. - Lets us check Monte Carlo decay and compare polynomial degrees.

0.3.24 Why this is the key validation for this folder

In `Original_Programs`, this is the principal refactored validation pathway: - compute weak errors, - visualize convergence behavior, - compare model quality versus sample size (and optionally versus degree).

So this step confirms whether the multi-level fitted surface preserves pricing expectations with acceptable error.

```
[6]: # Step 5: weak error validation for the multi-level fitted surface

# Validation grid
dt_val = h0 * 2**(-max_deg)
N_t_val = int(T / dt_val)

M_samples = [2000, 4000, 8000, 16000, 32000, 64000]
trials = 10

# Compute weak errors for current fitted b_bar_ml
weak_errors_ml = compute_weak_error_ML(
    b_bar=b_bar_ml,
    x0=x0,
    P1=P1,
    vol=vol,
    cov_mat=cov_mat,
    r=r,
    dt=dt_val,
    N_t=N_t_val,
    M_samples=M_samples,
    trials=trials
)

print("\n--- Step 5 summary (single degree) ---")
print("weak_errors_ml shape:", weak_errors_ml.shape)
print("mean weak error by M:", np.mean(weak_errors_ml, axis=1))
print("std weak error by M:", np.std(weak_errors_ml, axis=1, ddof=1))

# Plot in the function's expected dictionary format
weak_errors_dict = {max_deg: weak_errors_ml}
max_deg_test = [max_deg]
```

```

fig, ax = plot_weak_error_convergence(
    M_samples=M_samples,
    weak_errors_dict=weak_errors_dict,
    max_deg_test=max_deg_test,
    save_path='weakerror_ML_maxd3.pdf'
)

plt.show()

```

Completed 10 trials with M = 2000
 Completed 10 trials with M = 4000
 Completed 10 trials with M = 8000
 Completed 10 trials with M = 16000
 Completed 10 trials with M = 32000
 Completed 10 trials with M = 64000

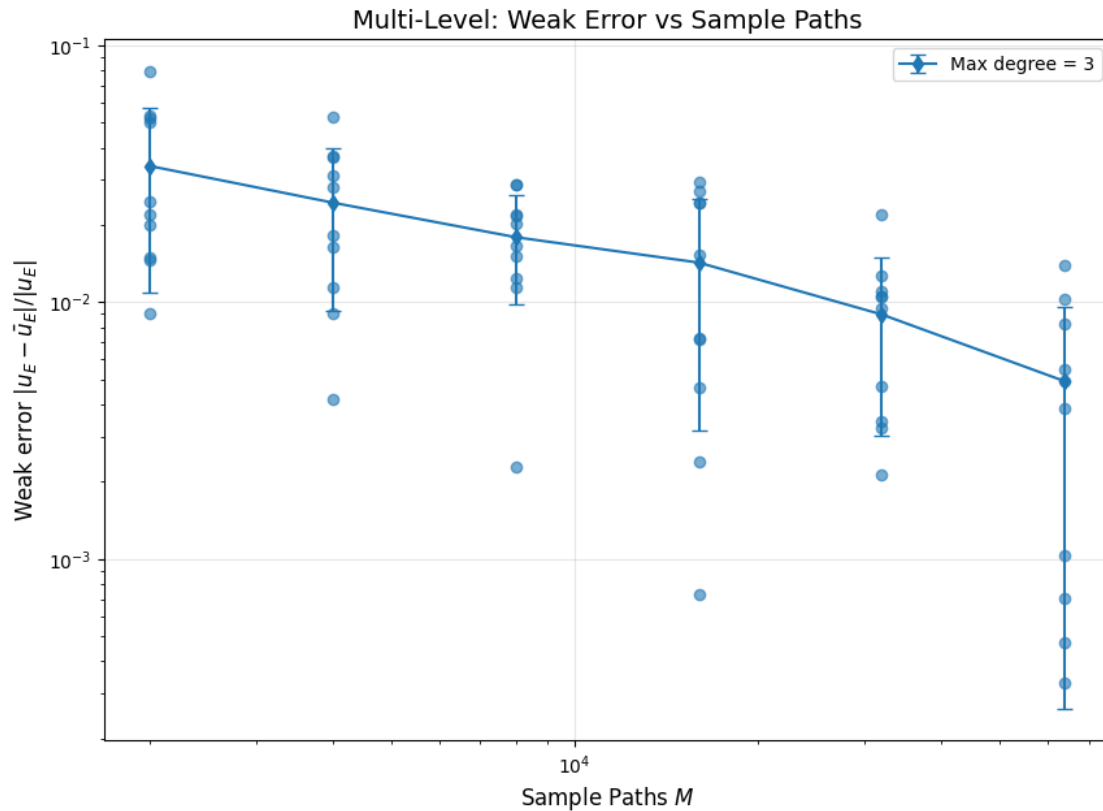
--- Step 5 summary (single degree) ---

weak_errors_ml shape: (6, 10)

mean weak error by M: [0.03395631 0.02444028 0.01792638 0.01427335 0.00896338
0.00492434]

std weak error by M: [0.02308789 0.01515129 0.00813873 0.01111988 0.00594361
0.00466184]

Figure saved to: weakerror_ML_maxd3.pdf



```

[8]: print(b_bar_ml(0.0, np.mean(basket0[:, 0])))
      print(b_bar_ml(T/2, np.mean(basket0[:, basket0.shape[1]//2])))
      print(b_bar_ml(T, np.mean(basket0[:, -1])))

dt_plot = h0 * 2**(-max_deg)
N_t_plot = int(T / dt_plot)
t_plot = np.linspace(0, T, N_t_plot)
fig, ax = plot_local_volatility(
    b_bar=b_bar_ml,
    basket=basket0,
    t=t_plot,
    K=50,
    L=150,
    title=f"ML Projected Volatility Surface (max_deg={max_deg})",
    color="C0",
    save_path='MLsurfmd3.pdf', # or e.g. "ML_VolSurf_maxdeg3.pdf"
)
plt.show()
print("basket0 shape:", basket0.shape)
print("len(t_plot):", len(t_plot))

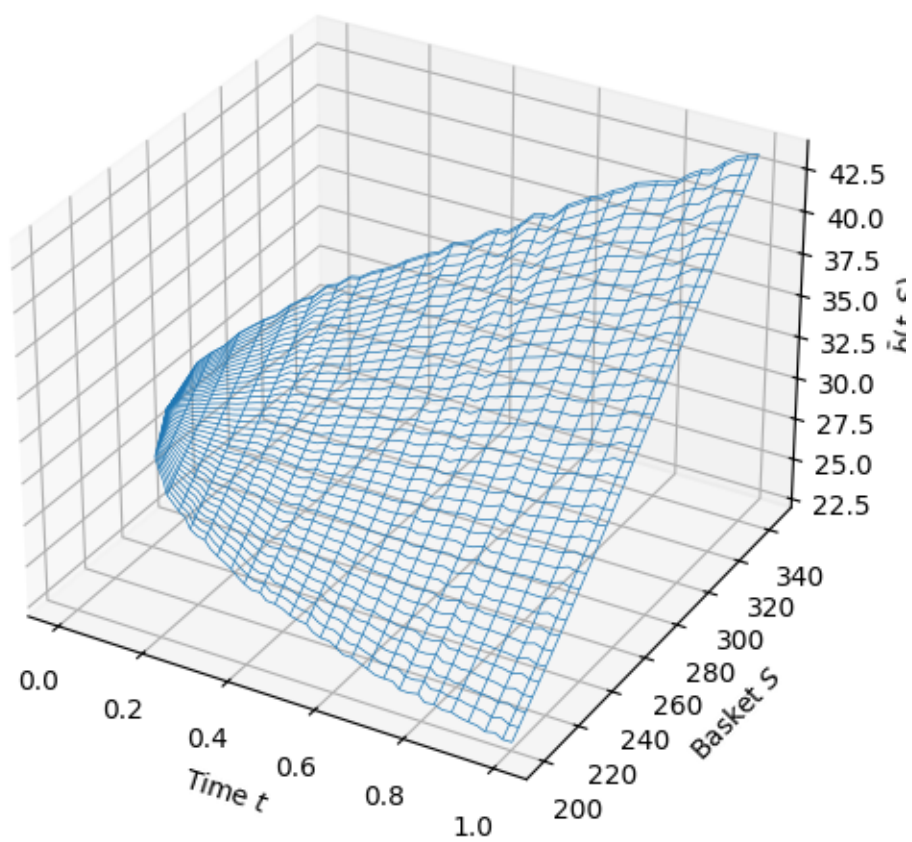
```

```

29.498846946194163
30.2845726314905
31.030344111280765

```

ML Projected Volatility Surface (max_deg=3)



```
basket0 shape: (10000, 64)  
len(t_plot): 64
```

```
[ ]:
```